

DepLens Architecture & Technical Specification

A Mathematical Software Supply Chain Intelligence & Cascading Blast Radius Engine

Problem Domain: Cybersecurity & Supply Chain Security	Architecture: Python/FastAPI Backend + React/TypeScript SVG
Mathematical Engine: Brandes Centrality, AST Parsing & Blast Radius	Optimization: Constrained Linear Cost Minimization (Residual Risk ≤ 30)

1. Executive Summary & Problem Formulation

Modern cloud applications depend on deeply nested software packages. Declaring just 10 direct libraries in a requirements.txt commonly pulls in **40–60 transitive dependencies** across 4–5 hierarchical depth levels, creating critical supply chain blind spots (e.g. Log4j, XZ Utils, colors.js).

Why Conventional Security Scanners Fail (Dependabot, Snyk, npm audit):

- **Flat-List Alert Fatigue:** Scanners output unranked lists of hundreds of CVEs without operational context, paralyzing engineers.
- **AST Source Code Blindness:** Existing tools treat a CVE in an unimported test script with the same urgency as a vulnerability in a core production request handler.
- **Absence of Graph Topology:** Scanners ignore hierarchical node depth, betweenness centrality, and cascading blast radius reach.
- **Destructive Remediation:** Tools mandate 'Update everything', creating catastrophic breaking changes and unbudgeted engineering costs.

The DepLens Breakthrough: DepLens transforms dependency trees into a formal **Directed Graph Network**, correlates live **OSV/NVD CVE databases**, parses real codebase Abstract Syntax Trees (AST) to pinpoint exact line numbers and endpoints, calculates cascading **Blast Radius (BR)** via reverse BFS, and solves a **Remediation Cost Optimization** model to achieve target risk reduction at minimal expense.

2. End-to-End System Processing Pipeline

Phase	Component	Functionality & Implementation Details
1	Manifest Ingestion	Parses direct requirements from requirements.txt, pyproject.toml, and package manifests.
2	Recursive Resolver	Recursively resolves full transitive sub-dependency hierarchy into directed graph edges.
3	NetworkX Engine	Constructs directed graph $G=(V,E)$, computes node degrees and max propagation depth.
4	Centrality Analyzer	Executes Brandes' algorithm to compute betweenness centrality $C_B(v)$ for all nodes.
5	AST Usage Scanner	Parses Python AST nodes, finding exact import statements, functions, endpoints, and lines.
6	Vulnerability Correlator	Correlates package versions against OSV.dev/NVD CVE feeds, computing CVSS Severity.
7	Blast Radius BFS	Executes reverse Breadth-First Search to calculate Dependency, Module, and Depth impacts.
8	Composite Risk Matrix	Synthesizes multi-factor risk score: 35% Sev + 30% BR + 20% Cent + 15% Usage.
9	Cost Optimization	Generates UPDATE, MITIGATE, REPLACE strategies solving $\min(\text{Cost})$ for $\text{ResidualRisk} \leq 30$.
10	Interactive Visualizer	Renders SVG canvas with pan/zoom, blast propagation halo, and multi-tab inspector.

3. Mathematical Formulations & Algorithmic Models

DepLens replaces arbitrary risk categorization with formal mathematical models across graph theory, static AST analysis, and constrained optimization:

3.1 Betweenness Centrality (C_B)

Betweenness Centrality quantifies how often a package v acts as a structural bridge along shortest paths between all other pairs of packages (s, t) in the directed graph $G = (V, E)$:

$$C_B(v) = \sum [\sigma_{st}(v) / \sigma_{st}] \times 100 \text{ (for all } s \neq v \neq t \text{)}$$

Where σ_{st} is the total number of shortest dependency paths from package s to t , and $\sigma_{st}(v)$ is the number of those paths that pass through v . Normalized on a 0–100 scale using NetworkX. High centrality packages (e.g. pydantic or typing-extensions) act as key bridges—if compromised, multiple independent subsystems are simultaneously exposed.

3.2 AST Code Usage Scoring

Static Abstract Syntax Tree (AST) analysis scans every source file in the repository to measure real code invocation. It inspects Import / ImportFrom nodes, maps aliased symbols, and traverses FunctionDef and endpoint decorators to assign scores based on operational buckets:

Source Code Reference Footprint	Score	Architectural Classification
0 Project Files (Transitive only / Uncalled)	10 / 100	Dormant / Edge Library
1 – 2 Project Files	30 / 100	Localized / Isolated Usage
3 – 5 Project Files	60 / 100	Moderate Multi-Module Usage
6 – 10 Project Files	80 / 100	Wide Architectural Dependency
> 10 Project Files	100 / 100	Core System Backbone / Framework

3.3 Cascading Blast Radius (BR)

Blast Radius quantifies the cascading damage across the dependency tree if a package fails or is compromised. It executes a reverse Breadth-First Search (BFS) starting from the target node:

$$BR = 50\% \times D + 30\% \times M + 20\% \times P$$

- **Dependency Impact (D):** $D = (\text{Affected Dependents} / \text{Total Dependencies}) \times 100$
- **Module Impact (M):** $M = (\text{Affected Source Code Files} / \text{Total Code Modules}) \times 100$
- **Propagation Depth Ratio (P):** $P = (\text{Reverse Traversal Depth } d / \text{Max Graph Depth } d_{\text{max}}) \times 100$

3.4 DepLens Composite Risk Score

The overall threat index is computed by synthesizing exploit severity, graph topology, code footprint, and cascading blast radius:

$$\text{Risk} = 35\% \times \text{Severity} + 30\% \times \text{BlastRadius} + 20\% \times \text{Centrality} + 15\% \times \text{CodeUsage}$$

Where **Severity** = $CVSS \times 10$ (0 if no active CVE). The resulting 0–100 score maps directly to actionable operational categories: **Low** (0–29), **Moderate** (30–49), **High** (50–69), **Critical** (70–84), and **Severe** (85–100).

3.5 Remediation Cost Optimization Model

Rather than mandating blind upgrades that break builds, DepLens solves a constrained cost-optimization model: finding the action with lowest engineering cost that brings residual risk below safe tolerance (**Residual Risk ≤ 30**):

$$\text{argmin}(\text{Cost}) \text{ subject to } \text{ResidualRisk} = \text{OriginalRisk} \times (1 - F) \leq 30$$

Strategy	Risk Reduction (F)	Engineering Effort & Cost Model	Typical Use-Case
UPDATE	F = 85%	Effort = Base + Version Drift (4–16 hrs) @ Rs. 1,000/hr	Clean minor/patch release available without API breakage.
MITIGATE	F = 55%	Effort = 2–4 hrs (Fixed Virtual Patch / WAF) @ Rs. 1,000/hr	Urgent zero-day / ReDoS where full upgrade requires extensive refactor.
REPLACE	F = 94%	Effort = 24–60 hrs (Complete migration) @ Rs. 1,000/hr	Abandoned / unmaintained package with critical unpatched CVEs.

4. Empirical Validation: FluxPay-server Case Study

We validated DepLens against GauthamSalian/FluxPay-server, a production payment orchestration and transaction gateway server. The pipeline generated the following empirical findings:

Topology Metric	Value	Vulnerability Metric	Value
Direct Manifest Packages	10	Active CVEs Identified	4
Transitive Sub-Dependencies	39	Max Risk Score in Tree	44 / 100 (Moderate)
Total Directed Edges	61	Average Tree Risk Score	10.3 / 100
Max Propagation Depth	4	AST Affected Source Files	5 Core Modules

Identified Security Vulnerabilities (CVE Audits):

Package	Version	CVE Identifier	CVSS	Fixed Version	Vulnerability Description & Attack Vector
fastapi	0.110.0	CVE-2024-24762	7.5 (High)	0.115.0	ReDoS in multipart form-data parser when handling malformed boundary headers.
sqladmin	0.16.1	CVE-2024-34064	8.7 (High)	0.19.0	Stored/Reflected XSS in admin model view column rendering via unescaped HTML.
starlette	0.36.3	CVE-2024-24768	7.1 (High)	0.37.2	Resource consumption / CPU Denial of Service during multipart upload parsing.
aiohttp	3.9.3	CVE-2024-27306	8.1 (High)	3.9.5	HTTP Request Smuggling via malformed chunk extensions in server parser.

4.1 AST Source Code References for fastapi (32 Usages across 5 Files):

File Path	Line Numbers	Import Statement & Invoked Endpoints	Type
app.py	L1, L7, L21–23	from fastapi import FastAPI, HTTPException app = FastAPI(title='FluxPay Server')	Module Init
api/walletroutes.py	L1, L18–35	from fastapi import APIRouter, Depends, HTTPException @router.post('/create'), @router.get('/balance')	Endpoints
api/transactionroutes.py	L1, L15–42	from fastapi import APIRouter, Depends @router.post('/transfer'), @router.post('/verify')	Endpoints
api/userroutes.py	L1, L12–28	from fastapi import APIRouter, Depends @router.post('/login'), @router.post('/register')	Endpoints
services/userservices.py	L1, L8–16	from fastapi import HTTPException raise HTTPException(status_code=400)	Functions

5. Commercialization, Monetization & Roadmap

To satisfy the 6 hackathon evaluation pillars (Innovation, Feasibility, Marketing, Monetization, Adherence, and Prototype), DepLens includes a robust commercialization framework:

5.1 Market Outreach & Go-To-Market Strategy (Pillar 3)

- **Developer-First Bottom-Up Adoption:** Free open-source CLI and GitHub Action generating PR-level blast radius comments.
- **Target Audiences:** DevSecOps teams, FinTech payment gateways, Healthcare SaaS, and Enterprise compliance teams requiring SBOM.
- **Content Marketing:** Publishing weekly 'Dependency Vulnerability Blast Radius' teardowns of top PyPI/npm packages.

5.2 Monetization Model & Financial Sustainability (Pillar 4)

Tier	Pricing	Target Customer	Features & Capabilities
Community	Free (Open Source)	Individual Developers & OSS	Single repo scans, web visualizer, standard OSV feed.
Team Pro	\$29 / repo / month	Mid-market Engineering Teams	Automated CI/CD PR blocking, AST code scanner, remediation optimizer.
Enterprise	\$499+ / org / month	Enterprises (Fintech/Health)	On-premises deployment, custom SLA, private registry support, compliance export.

5.3 Technical Roadmap & Milestones (Pillar 2)

- **Q1 2026 (Completed):** Core graph engine, NetworkX centrality, AST parser, reverse BFS blast radius, and interactive SVG drawer.
- **Q2 2026:** Real-time streaming OSV.dev & GitHub webhook integration, automated PR virtual patch generator.
- **Q3 2026:** Support for multi-language mono-repos (Rust Cargo, Go modules, Java Maven, npm/pnpm workspaces).
- **Q4 2026:** IDE plugin (VS Code & JetBrains) highlighting blast radius and CVE risks directly inside the code editor.

Conclusion: DepLens bridges the critical divide between raw vulnerability alerts and actual source code risk. By combining graph theory, static AST parsing, and cost optimization, it enables engineering organizations to protect their software supply chain efficiently without breaking builds.